

Onyx — FontKit reference

Onyx · multi-process OS on Raspberry Pi 4

The reference of **FontKit** (`SD:/lib/fontkit.so`, `user/Kits/fontkit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: [The Kits](#).

Contents

1. What it is
2. Using it
3. Index
4. `fontkit/uikitface.h`
5. `fontkit/fonts.h`

What it is

FontKit is fonts and text: FreeType as a shared library, the font manager (the card's families, a font at a size, its glyphs cached) and UIKit's anti-aliased text face.

Include	<code>#include "fontkit/uikitface.h"</code> <code>// or "fontkit/fonts.h"</code>
Link	<code>lib/fontkit.imp.a</code>
Library	<code>SD:/lib/fontkit.so</code> — 135 entries in its table (<code>user/Kits/fontkit/fontkit.abi</code> , append-only)
Sources	<code>user/Kits/fontkit/</code>

Using it

FontKit is FreeType for the applications, with the card's fonts (`SD:/res/fonts`).

Anti-aliased text in a UIKit application — one call before the widgets are built:

```
#include "uikit/uikit.h"
#include "fontkit/uikitface.h"
using namespace uikit;

int main (void)
{
    ft_uikit_install ("DejaVu Sans", 13);           // every widget's text from now
on
    Root root (400, 200, "Smooth text");
    root.addChild (new Label (12, 12, 376, 24, "FreeType draws this"));
    root.run ();
    return 0;
}
```

The font manager — the families installed, a font at a size, text measured (a word processor, a canvas):

```
#include "fontkit/fonts.h"
```

```
fnt::init ();  
for (int i = 0; i < fnt::count (); i++) ax_putln (fnt::name (i));
```

```
int fam = fnt::find ("DejaVu Serif");  
fnt::Font *f = fnt::get (fam, 0, 14 * 64);           // style 0 (regular), 14 px in  
26.6  
int width = fnt::advance (f, 'A');                  // in 26.6 units
```

FreeType's own functions

The library exports FreeType's API under its own names — 135 functions (FT_Init_FreeType, FT_New_Memory_Face, FT_Load_Glyph, FT_Render_Glyph...), used as FreeType's documentation says (freetype.org). The apps' build is lean: TrueType, the auto-hinter, the smooth rasterizer. Most programs do not call them: they use the font manager and the text face below.

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
FtTextFace	ft/uikitface.h -- FreeType's anti-aliased text for every uikit widget	uikitface.h
ft_uikit_install	Installs FreeType's text for every uikit widget with the family by name (DejaVu Sans when absent) at px pixels, false when the card has no TrueType font.	uikitface.h
ft_uikit_face	The face ft_uikit_install made (0	uikitface.h
ft_dialog_face	The face of the dialogs (0	uikitface.h
ft_messagebox	uikit's message box (uk_messagebox) in FreeType's text.	uikitface.h
ft_file_open	uikit's file dialogs (uk_file_open / uk_file_save / uk_folder_open) in FreeType's text.	uikitface.h
ft_file_save	uikit's file dialogs (uk_file_open / uk_file_save / uk_folder_open) in	uikitface.h

Name	What it does	Header
	FreeType's text.	
<code>ft_folder_open</code>	UIKit's file dialogs (<code>uk_file_open</code> / <code>uk_file_save</code> / <code>uk_folder_open</code>) in FreeType's text.	<code>UIKitface.h</code>
<code>Glyph</code>	(a type)	<code>fonts.h</code>
<code>Font</code>	(a type)	<code>fonts.h</code>
<code>FaceFile</code>	(a type)	<code>fonts.h</code>
<code>s_len</code>	A string's length in bytes (0 for a null pointer).	<code>fonts.h</code>
<code>s_cpy</code>	Copies <code>s</code> into <code>d</code> , cut to <code>cap - 1</code> bytes and always ended by a 0 (a null <code>s</code> gives an empty string).	<code>fonts.h</code>
<code>lc</code>	An ASCII capital letter as its small letter, anything else unchanged.	<code>fonts.h</code>
<code>s_icmp</code>	Compares two strings with ASCII letters' case ignored, returning 0 when equal, else < 0 or > 0 as <code>strcmp</code> .	<code>fonts.h</code>
<code>be16</code>	The big-endian 16-bit value at <code>p</code> .	<code>fonts.h</code>
<code>be32</code>	The big-endian 32-bit value at <code>p</code> .	<code>fonts.h</code>
<code>read_at</code>	Reads <code>n</code> bytes of the open file <code>f</code> at offset <code>pos</code> into <code>buf</code> , false when the seek fails or fewer bytes come.	<code>fonts.h</code>
<code>scan_file</code>	A font file's family name and style from its own tables (no FreeType)	<code>fonts.h</code>
<code>is_ttf</code>	True when the file name <code>n</code> ends in ".ttf" (letters' case ignored).	<code>fonts.h</code>
<code>scan_dir</code>	Adds every .ttf file of the folder <code>dir</code> that <code>scan_file</code> accepts to <code>g_face</code> (MAXFACE files at most).	<code>fonts.h</code>

Name	What it does	Header
<code>add_to_family</code>	The style slot a file fills in its family	<code>fonts.h</code>
<code>init</code>	Starts FreeType and sorts the fonts of SD:/res/fonts and SD:/fonts into families (once), false when FreeType fails or no TrueType font is there.	<code>fonts.h</code>
<code>count</code>	The number of families <code>init ()</code> found.	<code>fonts.h</code>
<code>name</code>	Family i's name, "" when there is no such family.	<code>fonts.h</code>
<code>find</code>	A family's index by its name (letters' case ignored), -1 when there is none.	<code>fonts.h</code>
<code>styles</code>	The styles a family has (a mask of $1 << \text{style}$).	<code>fonts.h</code>
<code>open_face</code>	A face file's FreeType face, the file read at its first use.	<code>fonts.h</code>
<code>free_glyphs</code>	Frees a font's glyph table and its bitmaps and lowers the cache's byte count (the Font itself stays).	<code>fonts.h</code>
<code>make</code>	Builds a family's style at size64 (1/64 px, 1 px at least) with its metrics and an empty glyph table, 0 when the family or its face file fails.	<code>fonts.h</code>
<code>get</code>	A family's style at a size (1/64 px).	<code>fonts.h</code>
<code>void</code>	Forget the fonts not used lately when the cache holds too much (a Font from <code>get ()</code> is then stale)	<code>fonts.h</code>
<code>trim</code>	Forget the fonts not used lately when the cache holds too much (a Font from <code>get ()</code> is then stale)	<code>fonts.h</code>
<code>slot</code>	The glyph table's slot	<code>fonts.h</code>

Name	What it does	Header
	for the character cp, either the one holding it or the empty one where it goes.	
grow	Doubles the glyph table and puts its glyphs back in their new slots (earlier Glyph pointers are stale).	fonts.h
glyph	A character's cached glyph entry (0 is read as a space), made at its first use with its glyph index and advance.	fonts.h
advance	A character's advance in 1/64 px, 0 without a font.	fonts.h
kern	The kerning between the characters a and b in 1/64 px, 0 when there is none (no font, no kerning table, a glyph from the fallback font).	fonts.h
render	The coverage bitmap of a glyph at a quarter-pixel position.	fonts.h
blend	Lays the colour c (0xRRGGBB) over the pixel d at opacity a (0..255) keeping d's top byte, or replaces d by c when a >= 255.	fonts.h
draw	A character at x64 (1/64 px) on the baseline y, in colour c, clipped to [cx0, cx1) x [cy0, cy1).	fonts.h
draw_str	A string (Latin-1 / ASCII) at x, baseline y, the pen's end returned (1/64 px).	fonts.h
str_w	A string (Latin-1 / ASCII) at x, baseline y, the pen's end returned (1/64 px).	fonts.h

fontkit/uikitface.h

ft/uikitface.h -- FreeType's anti-aliased text for every uikit widget: a uikit::TextFace (uikit/text.h) built on ft/fonts.h (the card's TrueType families, FreeType's glyphs cached per quarter-pixel position). Header-only, like ft/fonts.h: include it once, in the app's one translation unit, and link ft/libft.a -- a newlib app (Letters' rule in user/Makefile is the model).

```
#include "fontkit/uikitface.h"
int main () {
    ft_uikit_install ("DejaVu Sans", 13);    // before building the widgets;
false: none (bitmap)
    Root root (...); ...
}
```

Text is UTF-8 (a stray byte is read as Latin-1); styles 0 regular, 1 italic, 2 bold, 3 bold italic (a style the family lacks is made by fnt::). The line height is the font's (DejaVu Sans at 13 px: 16 px, as the bitmap font's), the text centred in it. More faces (a display's large digits):

```
FtTextFace *big = new FtTextFace; big->open ("DejaVu Sans", 24); lcd->face =
big;
```

fnt::trim () may run at any moment no fnt::Font pointer is held: the face holds none between calls.

```
class FtTextFace : public uikit::TextFace
{
public:
    bool kerning;           // the pairs' kerning (on)

    FtTextFace () : kerning (true), m_fam (-1), m_px (0), m_h (16), m_asc (12)
{ reset (); }

    // The family by name (falls back to DejaVu Sans), px the size (the em, in
pixels). fnt::init ()
// is called here. False: no TrueType font on the card.
    bool open (const char *family, int px)
    {
        if (!fnt::init ()) return false;
        int fam = family ? fnt::find (family) : -1;
        if (fam < 0) fam = fnt::find ("DejaVu Sans");
        if (fam < 0) fam = 0;
        if (px < 4) px = 4;
        m_fam = fam; m_px = px;
        reset ();
        fnt::Font *f = font (0);
        if (!f) { m_fam = -1; return false; }
        int glyphs = f->ascent + f->descent;           // (1/64 px)
        m_h = (f->height + 63) >> 6;
        if (m_h * 64 < glyphs) m_h = (glyphs + 63) >> 6;
        m_asc = ((m_h * 64 - glyphs) / 2 + f->ascent + 32) >> 6;    // the glyphs
centred in the line
        return true;
    }
    int family () const { return m_fam; }
    int px () const { return m_px; }

    int height () override { return m_h; }
    int ascent () override { return m_asc; }
    int width (const char *s, int style) override { return widthN (s, 1 << 30,
style); }
```

```

// The first n bytes' width (up to a NUL): the advances and kerning in 1/64 px,
rounded -- the
// same pen draw () moves. Remembered in a small cache (a label is measured at
each redraw).

```

```

int widthN (const char *s, int n, int style) override
{
    if (!s || n <= 0 || m_fam < 0) return 0;
    style &= 3;
    unsigned h = 2166136261u; int len = 0;
    for (; len < n && s[len]; len++) h = (h ^ (unsigned char) s[len]) *
16777619u;
    if (len == 0) return 0;
    h ^= (unsigned) style * 0x9E3779B9u;
    WCache &c = m_wc[(h ^ (h >> 16)) & (WC - 1)];
    if (c.len == len && c.hash == h && c.style == style) return c.w;
    fnt::Font *f = font (style);
    if (!f) return 0;
    long x64 = 0; unsigned prev = 0;
    for (int i = 0; i < len; )
    {
        int k; unsigned cp = uikit::uk_u8_get (s + i, len - i, &k); i += k;
        if (kerning && prev) x64 += fnt::kern (f, prev, cp);
        x64 += adv (f, style, cp);
        prev = cp;
    }
    int w = (int) ((x64 + 32) >> 6);
    c.hash = h; c.len = len; c.style = style; c.w = w;
    return w;
}

```

```

void draw (uikit::Canvas &cv, int x, int yTop, const char *s, unsigned color, int
style) override

```

```

{
    if (!s || !*s || m_fam < 0) return;
    style &= 3;
    fnt::Font *f = font (style);
    if (!f) return;
    int base = yTop + m_asc, n = 0;
    while (s[n]) n++;
    long x64 = (long) x * 64; unsigned prev = 0;
    color &= 0x00FFFFFFu;
    for (int i = 0; i < n; )
    {
        int k; unsigned cp = uikit::uk_u8_get (s + i, n - i, &k); i += k;
        if (kerning && prev) x64 += fnt::kern (f, prev, cp);
        if (x64 >= (long) cv.w * 64) break; // (the rest is past the
edge)
        fnt::draw (cv, f, (int) x64, base, cp, color, 0, 0, cv.w, cv.h);
        x64 += adv (f, style, cp);
        prev = cp;
    }
}

```

```

private:

```

```

    enum { WC = 128 };
    struct WCache { unsigned hash; int len, style, w; };
    int m_fam, m_px, m_h, m_asc;
    int m_adv[4][128]; // the ASCII advances per style, 1/64 px (-1: not yet
asked)
    WCache m_wc[WC];

```

```

void reset ()
{
    for (int s = 0; s < 4; s++) for (int c = 0; c < 128; c++) m_adv[s][c] = -1;
    for (int i = 0; i < WC; i++) { m_wc[i].len = -1; m_wc[i].hash = 0; }
    m_wc[i].style = 0; m_wc[i].w = 0; }
}
// uikit's style (1 italic, 2 bold) -> fnt's (BOLD 1, ITALIC 2); the sized font
asked each time
// (no pointer kept: fnt::trim may drop it between two calls).
fnt::Font *font (int style)
{
    int fs = ((style & 2) ? fnt::BOLD : 0) | ((style & 1) ? fnt::ITALIC : 0);
    return fnt::get (m_fam, fs, m_px * 64);
}
int adv (fnt::Font *f, int style, unsigned cp)
{
    if (cp < 128)
    {
        int &a = m_adv[style][cp];
        if (a < 0) a = fnt::advance (f, cp);
        return a;
    }
    return fnt::advance (f, cp);
}
};

```

Install FreeType's text for every uikit widget: the family by name ("DejaVu Sans", "Liberation Sans", "Selawik"..., DejaVu Sans when absent) at px pixels. False: no TrueType font (uikit keeps its bitmap fonts). Call it before building the widgets.

```
static FtTextFace *g_ftUIKitFace; // (the one ft_uikit_install made)
```

Installs FreeType's text for every uikit widget with the family by name (DejaVu Sans when absent) at px pixels, false when the card has no TrueType font.

```
bool ft_uikit_install (const char *family, int px);
```

The face ft_uikit_install made (0: none yet).

```
FtTextFace *ft_uikit_face ();
```

uikit's own dialogs in FreeType's text, in a program that does NOT install a face for all its widgets (Letters, Slides: their own interface has the bitmap text's metrics): the face is in use for the dialog's time only -- a message box, the file dialog, with their buttons and fields.

```
ft_messagebox (title, text, MB_YESNO); ft_file_open (path, sizeof path, dir);
```

A program that called ft_uikit_install has that face already: these then do as uikit's.

```
static FtTextFace *g_ftDialogFace; // (made at the first dialog: DejaVu Sans, 13
px)
```

The face of the dialogs (0: no TrueType font on the card -- the dialog keeps the bitmap text).

```
FtTextFace *ft_dialog_face ();
```

uikit's message box (uk_messagebox) in FreeType's text.

```
int ft_messagebox (const char *title, const char *text, int buttons);
```

uikit's file dialogs (uk_file_open / uk_file_save / uk_folder_open) in FreeType's text.

```
bool ft_file_open (char *out, unsigned cap, const char *startDir, const char *filters
= 0);
bool ft_file_save (char *out, unsigned cap, const char *startDir, const char
*defName, const char *filters = 0);
bool ft_folder_open (char *out, unsigned cap, const char *startDir);
```

fontkit/fonts.h

ft/fonts.h -- the apps' TrueType text (Letters' pages): the fonts on the card by family, at any size, their glyphs rendered by FreeType (user/Kits/fontkit/: TrueType only, anti-aliased, auto- hinted vertically) and cached. Header-only: include it once, in the app's one translation unit; link ft/libft.a (a newlib app: FreeType wants a libc).

fnt::init ();	the families of SD:/res/fonts and SD:/fonts
(sorted)	
fnt::count (), fnt::name (i)	... their names ("Liberation Serif", "DejaVu Sans Mono"...)
fnt::find ("Liberation Sans")	a family by name (-1: none)
fnt::Font *f = fnt::get (fam, fnt::BOLD fnt::ITALIC, size64)	a family's style at a size in 1/64 px (a style
the family	
f->ascent, f->descent, f->height	lacks is made: bold thickened, italic slanted)
the next)	its metrics, 1/64 px (height: from a line to
fnt::advance (f, cp)	a character's advance, 1/64 px (the design's:
exact at any	
fnt::kern (f, a, b)	size, so a line breaks the same at every zoom)
fnt::draw (cv, f, x64, y, cp, c, clip...)	the pair's kerning, 1/64 px
pixel)	a character at x (1/64 px: 4 positions a
	on the baseline y, in colour c, blended over
the canvas	
fnt::trim ()	drop the least used sizes when the cache is big
(at a	
	moment no Font pointer is held: after a redraw)

A character a font lacks is taken from DejaVu Sans (the card's widest one), same style.

```
enum { BOLD = 1, ITALIC = 2 };
enum { MAXFACE = 96, MAXFAM = 48, MAXFONT = 512 };
```

struct Glyph

```
{
    unsigned cp;           // the character (0: an empty slot)
    int gi;                // its glyph in src's face (0: none, a box drawn)
    int adv;               // advance, 1/64 px
    struct Font *src;       // the font it comes from (this one or the fallback)
    unsigned char *bmp[4];  // coverage bitmaps, one per quarter-pixel position
    (lazily)
    short bl[4], bt[4], bw[4], bh[4]; // their left / top (from the pen) / size
};
```

struct Font

```
{
    int fam, style, size64;
    int face;              // g_face[] index used
    bool fakeBold, fakeItalic; // the style is made
    FT_Size size;
```

```

    int ascent, descent, height;    // 1/64 px (descent positive, below the baseline)
    int ulPos, ulThick;             // underline: its top below the baseline, its thickness
(1/64 px)
    int xHeight;                    // the x-height (strike-through line: at its half)
    int boldX;                      // a made bold's extra advance
    Glyph *tab; int cap, n;         // the glyphs (open addressing on cp)
    Font *fallback;                 // DejaVu Sans at the same style and size (lazily; 0:
none)
    bool fbTried;
    unsigned used;                  // last get () (the least used go first)
    unsigned bytes;                 // its bitmaps' size
};

```

```

struct FaceFile
{
    char path[112];
    char family[48];
    int style;                      // BOLD / ITALIC from the file's own tables
    int weight;                     // OS/2 usWeightClass (400 regular, 700 bold)
    unsigned char *data; unsigned len;
    FT_Face face;
    bool failed;
};

```

```

struct Family { char name[48]; int file[4]; }; // file[style]: g_face[] index, -1
none

```

state

```

static FT_Library g_lib;
static bool g_init, g_ok;
static FaceFile g_face[MAXFACE]; static int g_nface;
static Family g_fam[MAXFAM]; static int g_nfam;
static Font *g_font[MAXFONT]; static int g_nfont;
static unsigned g_tick, g_bytes;
static int g_fallbackFam = -1;
static unsigned char g_gamma[256]; // coverage -> opacity (a slightly heavier stroke
on screen)

```

small helpers

A string's length in bytes (0 for a null pointer).

```
int s_len (const char *s);
```

Copies s into d, cut to cap - 1 bytes and always ended by a 0 (a null s gives an empty string).

```
void s_cpy (char *d, const char *s, int cap);
```

An ASCII capital letter as its small letter, anything else unchanged.

```
int lc (int c);
```

Compares two strings with ASCII letters' case ignored, returning 0 when equal, else < 0 or > 0 as strcmp.

```
int s_icmp (const char *a, const char *b);
```

The big-endian 16-bit value at p.

```
unsigned be16 (const unsigned char *p);
```

The big-endian 32-bit value at p.

```
unsigned be32 (const unsigned char *p);
```

Reads n bytes of the open file f at offset pos into buf, false when the seek fails or fewer bytes come.

```
bool read_at (void *f, unsigned pos, void *buf, unsigned n);
```

A font file's family name and style from its own tables (no FreeType: 3 small reads). False: not a TrueType font.

```
bool scan_file (const char *path, FaceFile &ff);
```

True when the file name n ends in ".ttf" (letters' case ignored).

```
bool is_ttf (const char *n);
```

Adds every .ttf file of the folder dir that scan_file accepts to g_face (MAXFACE files at most).

```
void scan_dir (const char *dir);
```

The style slot a file fills in its family: the one it says; a family with two files for one slot (a light and a regular...) keeps the one nearest the slot's weight.

```
void add_to_family (int fi);
```

the families

Starts FreeType and sorts the fonts of SD:/res/fonts and SD:/fonts into families (once), false when FreeType fails or no TrueType font is there.

```
bool init ();
```

The number of families init () found.

```
int count ();
```

Family i's name, "" when there is no such family.

```
const char *name (int i);
```

A family's index by its name (letters' case ignored), -1 when there is none.

```
int find (const char *n);
```

The styles a family has (a mask of 1 << style).

```
int styles (int fam);
```

A face file's FreeType face, the file read at its first use.

```
FT_Face open_face (int fi);
```

sized fonts

Frees a font's glyph table and its bitmaps and lowers the cache's byte count (the Font itself stays).

```
void free_glyphs (Font *f);
```

Builds a family's style at size64 (1/64 px, 1 px at least) with its metrics and an empty glyph table, 0 when the family or its face file fails. A style the family lacks is made from another of its files (bold thickened, italic slanted).

```
Font *make (int fam, int style, int size64);
```

A family's style at a size (1/64 px).

```
Font *get (int fam, int style, int size64);
```

Forget the fonts not used lately when the cache holds too much (a Font from get () is then stale: only at a moment no Font pointer is held -- after a redraw; the app forgets its own, g_onTrim).

```
static void (*g_onTrim) ();  
void trim (unsigned maxBytes = 6u << 20, int maxFonts = 96);
```

glyphs

The glyph table's slot for the character cp, either the one holding it or the empty one where it goes.

```
Glyph *slot (Font *f, unsigned cp);
```

Doubles the glyph table and puts its glyphs back in their new slots (earlier Glyph pointers are stale).

```
void grow (Font *f);
```

A character's cached glyph entry (0 is read as a space), made at its first use with its glyph index and advance. A character the font lacks comes from DejaVu Sans (nowhere either: gi 0, half the size wide). The pointer holds until the table grows.

```
Glyph *glyph (Font *f, unsigned cp);
```

A character's advance in 1/64 px, 0 without a font.

```
int advance (Font *f, unsigned cp);
```

The kerning between the characters a and b in 1/64 px, 0 when there is none (no font, no kerning table, a glyph from the fallback font).

```
int kern (Font *f, unsigned a, unsigned b);
```

The coverage bitmap of a glyph at a quarter-pixel position.

```
void render (Font *f, Glyph *g, int ph);
```

Lays the colour c (0xRRGGBB) over the pixel d at opacity a (0..255) keeping d's top byte, or replaces d by c when a >= 255.

```
void blend (unsigned &d, unsigned c, int a);
```

A character at x64 (1/64 px) on the baseline y, in colour c, clipped to [cx0, cx1) x [cy0, cy1).

```
void draw (UIKit::Canvas &cv, Font *f, int x64, int y, unsigned cp, unsigned c, int cx0, int cy0, int cx1, int cy1);
```

A string (Latin-1 / ASCII) at x, baseline y, the pen's end returned (1/64 px). For labels.

```
int draw_str (UIKit::Canvas &cv, Font *f, int x64, int y, const char *s, unsigned c);  
int str_w (Font *f, const char *s);
```